JS Functions 💡

7 JavaScript Functions Pro Developers Keep Secret 🔥

Are you still writing **20 lines** of code for something that could be done in **1 Line** ?

@anantshah133⚡

1 Array.flatMap()

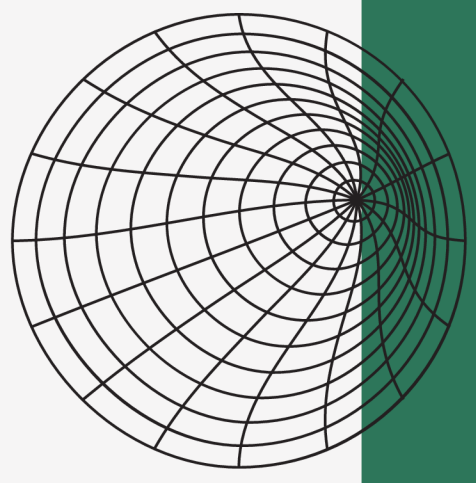
Problem : Mapping an array and then flattening the result is a common pattern that usually requires two operations.

Pro Solution : flatMap() processes each element through a mapping function, then flattens the result into a new array.

```
let array = [1, 2, 3];

// ❌ Instead of this:
const result = array.map(x => [x, x * 2]).flat();

// ✅ Do this:
const result = array.flatMap(x => [x, x * 2]);

// 🔸 Output : [1, 2, 2, 4, 3, 6]
```

@anantshah133⚡

2 Object.fromEntries()



Problem : Converting arrays of key-value pairs into objects requires manual iteration.

Pro Solution : **Object.fromEntries()** transforms a list of key-value pairs into an object in one line.

```
const entries = [['name', 'John'], ['age', 30]];

// 🪄 One line magic:
const person = Object.fromEntries(entries);

// 🔸 Output : { name: 'John', age: 30 }
```

@anantshah133⚡

3 String.replaceAll()

Problem : Replacing all occurrences of a substring used to require regex with the global flag.

Pro Solution : The **replaceAll()** method makes this clean and intuitive.

```
const message = "This is a test. This needs fixing.";

// ❌ Old way:
message.replace(/This/g, 'That');

// ✅ Pro way:
message.replaceAll('This', 'That');

// 🔸 Output : "That is a test. That needs fixing."
```

@anantshah133⚡

4 Array.at() 📌

↘

Problem : Getting elements from the end of an array is cumbersome with traditional indexing.

Pro Solution : The **at()** method accepts negative integers, counting from the end.

```
1  const team = ['John', 'Mary', 'Bob', 'Alice'];
2
3  // ❌ Instead of this :
4  team[team.length - 1]; // 'Alice'
5
6  // 🪄 Pro developers do:
7  team.at(-1); // 'Alice'
8  team.at(-2); // 'Bob'
```

@anantshah133⚡

5

Promise.allSettled() 🧘‍♂️

↘

Problem : When handling multiple promises, one rejection causes **Promise.all()** to fail completely.

Pro Solution : **Promise.allSettled()** waits for all promises regardless of fulfilled or rejected state.

```
const promises = [
    fetch('/data-1'),
    fetch('/data-2'),
    fetch('/might-fail')
];

// 🪄 Get results even if some fail:
Promise.allSettled(promises).then(results => {
    // All results available here, including rejected ones
});
```

@anantshah133⚡

6 Optional Chaining (?.)

Problem : Deeply nested object properties cause errors if any intermediate property is null/undefined.

Pro Solution : **Optional chaining (?.)** prevents these errors elegantly.

```
let user = {
    profile: {
        name: "John Doe"
    },
};

// ❌ Instead of this :
const name = user && user.profile && user.profile.name;

// ✅ Pro way :
const name = user?.profile?.name;

// 🔸 Output : John Doe
```

@anantshah133⚡

7 Array.find() 🕵️

↘

Problem : Using loops or filters to find the first matching element is verbose and inefficient.

Pro Solution : Use **Array.find()** to directly return the first element that matches a condition.

```
const users = [
    { id: 1, name: "John" },
    { id: 2, name: "Alice" },
    { id: 3, name: "Bob" }
];

// ❌ Old way (using filter + [0] or loops):
const userOld = users.filter(user => user.id === 2)[0];

// ✅ Pro way:
const userPro = users.find(user => user.id === 2);

console.log(userPro);
//  🔸 Output: { id: 2, name: 'Alice' }
```

@anantshah133⚡

Thank You... 🌟

↘ Save

Level up beyond YouTube tutorials! 💪

💡 Hope you found these JS tips useful !

📌 Save this post so you never forget these pro moves.

💬 Comment your fav one-liner from the list.

Follow Me :

@anantshah133⚡

👋